

화면을 띄우는 것이 가장 강한 시험이었습니다

스크린샷을 찍으려고 브라우저를 열자 이벤트 조회 라우트가 500이었습니다. 단위 시험 530건이 전부 초록인 채로 그 라우트는 운영에서 죽어 있었습니다. 3장을 찍으며 죽은 라우트 하나를 잡았습니다 — 16장이면 더 나옵니다.

D-386 · 값이 반대편에 있었습니다

스크린샷은 「보여줄 것」이 아니라
「우리가 못 보던 것을 보는 눈」입니다

단위 시험은 함수를 부르고, 브라우저는 라우트를 때립니다. 그 사이에 URL 배선·직렬화·권한·미들웨어 순서·정적 자산이 다 들어 있고 단위 시험은 그 전부를 건너뛸니다. 그래서 시험 자체가 브라우저를 열도록 바꿉니다 — 캡처는 부산물이고 본체는 시험입니다.

68.2%

상용 · 정정 실제 대비 +2.9%p

0 → 3/16

스크린샷 · 결함 1건 발견

314건

호출 없음 · 관찰 198

v3.7 → v3.8

195 → 203

직전 67.3%를 65.3%로 정정합니다

직전 보고 67.3% 거짓 초록 둘을 포함	검어낸 실제 65.3% ④운영 60→48 · ①기능 83→82	지금 68.2% 정정 실제 대비 +2.9%p
--	---	---

영역	가중	직전 보고	정정 실제	지금	변화	잠재
① 기능 완결성	20%	83%	82%	84%	+2%p	+3.2%p
② 보안	20%	68%	68%	68%	-	+6.4%p
③ 격리·다중테넌시	10%	88%	88%	88%	-	+1.2%p
④ 운영	15%	60%	48%	62%	+14%p	+5.7%p
⑤ 성능·확장	10%	45%	45%	45%	-	+5.5%p
⑥ 품질	10%	84%	84%	85%	+1%p	+1.5%p
⑦ 사용성	10%	55%	55%	58%	+3%p	+4.2%p
⑧ 법·인증	5%	18%	18%	18%	-	+4.1%p

§ 거짓 초록 둘 — 도구가 자기를 만든 턴의 보고를 물었습니다

거짓 초록 ① 「주기를 등록했다」가 실제로는 돌지 않고 있었습니다

test_every_beat_entry_points_at_a_registered_task가 첫 실행에 세 줄을 뱉었습니다. celery는 앱의 tasks.py만 읽는데 우리 태스크는 ops_tasks.py에 있었습니다.

→ 감시·백업이 「등록됨」으로 보고되고 실제로는 돌지 않았습니다. 고침은 파일 하나였습니다.

거짓 초록 ② 단위 530건 전부 초록인데 라우트는 운영에서 500

캡처 첫 시도에서 /api/dsm/events가 500이었습니다. 스크린샷을 찍으려 하지 않았으면 영원히 몰랐을 것입니다.

★ 「컨 것이 도는 것을 시험이 본다」는 지시가 한 턴 만에 값을 했습니다. verify_dormant가 자기를 만든 턴의 보고를 물었고, 그것이 옳았습니다.

원칙 신설 — 「켰다」·「등록했다」·「연결했다」는 보고는 「돌았다」의 증거를 함께 냅니다. 등록 목록에 이름이 있는 것은 증거가 아닙니다 — 실제 실행 로그 1건 · 실제 응답 1건 · 실제 발화 1건입니다.

값이 반대편에 있었습니다

"캡처 첫 시도에서 `/api/dsm/events`가 500이었습니다 —
단위 시험 530건이 전부 초록인 채로 그 라우트는 운영에서 죽어 있었습니다."

단위 시험은 함수를 부르고, 브라우저는 라우트를 때립니다. 그 사이에 URL 배선 · 직렬화 · 권한 · 미들웨어 순서 · 정적 자산이 다 들어 있고, 단위 시험은 그 전부를 건너뜁니다.
이건 착시 ⑨(배선)의 가장 순수한 형태입니다 — 코드도 있고 시험도 초록인데 운영에서 죽어 있습니다.

★ 그래서 스크린샷의 우선순위를 올립니다. 지금까지 저는 이것을 「고객에게 보여줄 것」으로 다뤘습니다. 실제로는 「우리가 못 보던 것을 보는 유일한 눈」이었습니다 — 값이 반대편에 있었습니다.
3장을 찍으며 죽은 라우트 1건, 16장이면 더 나옵니다.

지시 E2E가 브라우저를 연다 — 캡처는 부산물, 본체는 시험

D-347 ③ 그대로

지금 E2E 23종은 API 수준이라 이 층을 못 봅니다. 화면이 있는 시나리오 단계는 브라우저로 때리고 캡처합니다.
500·404·빈 화면이 나오면 그것이 이번 턴의 발견입니다. 고치고 다시 찍습니다 — 에러를 지운 화면이 아니라 에러가 나지 않는 상태에서 찍은 화면이어야 합니다.

`verify_route_alive.py` — 화면이 쓰는 라우트 전수를 실제 HTTP로 때려 2xx 확인. ★ D-310 자기표본: 이번엔 500을 내던 그 라우트를 fixture로. ★ 이 게이트는 단위 시험이 초록이어도 독립적으로 죽을 수 있어야 합니다 — 그것이 존재 이유입니다.

§ D-387 · 상용 출시를 막는 것 — 빌드가 재현되지 않습니다

`ry-core`가 커밋이 아니라 브랜치를 가리킵니다. 「저장소가 정본」을 적용할 대상 자체가 없고, 그것이 타입오류 3,229건의 뿌리입니다. 제가 D-381 ④에 「lock이 없거나 어긋나면 그것이 진짜 결함」이라 적었고 — 없는 쪽이었습니다.

문제의 크기	왜 상용에 못 나가나
시점마다 다른 코드	브랜치를 가리키면 같은 명령이 시점마다 다른 코드를 가져옵니다. 오늘 되던 빌드가 내일 깨지고, 원인은 우리 커밋이 아니라 남의 브랜치가 움직인 것입니다
배포마다 다른 것	배포할 때마다 다른 것이 나갈 수 있습니다. 이건 상용 출시가 불가능한 상태입니다
재현성 원칙	D-344(측정 재현성)의 빌드 판 — 두 번 돌려 다른 것이 나오면 그건 빌드가 아닙니다

- 1 지금 설치된 `ry-core`의 커밋 해시를 실측하라 — 그것이 오늘 우리가 서 있는 땅이다
- 2 그 해시로 고정한다(lock 생성 또는 커밋 핀). 브랜치 참조를 남기지 마라
- 3 고정 후 깨끗한 곳에서 빌드 1회 — 같은 결과가 나오는지. 다르면 고정이 안 된 것
- 4 다른 브랜치 참조 의존성이 더 있는지 전수하고 건수 출력. 나는 그 수를 모른다

고칠 목록이 아니라 가를 목록입니다

슬어	모수	잡든 것	고치는 법	이번 턴
㉠ 호출 없음	1,550	314건	배선	가르기만. ㉡ 목록을 나에게
㉡ 주기 없음	41	11 → 10건	등록	고쳐짐 — 다만 돌지 않던 것을 이번에 잡음
㉢ 설정 빔	30	2건	데이터	고쳐짐
합계 326 · \$0.4 128(잔여 분모 제외) · 우리 관할 198 · ㉢은 접지 못한 13건을 모수 밖으로 — 「모른다」이지 「비지 않았다」가 아님				

★ 314를 그대로 두면 큰 수 하나가 남을 뿐입니다. 세 갈래로 가릅니다.

㉠ **켜야 할 것**

만들어졌고 쓸모가 있는데 **배선이 없다**.
오늘의 자산입니다 — 이미 만든 것을 켜는 건 새로 만드는 것보다 싸니다

㉡ **지워야 할 것**

대체되었거나 쓰이지 않는 **죽은 코드**.
오늘의 빛입니다 — 다음 사람이 「이건 뭐가」를 묻습니다

㉢ **정상**

유틸·헬퍼·예외 경로·시험 전용.
잡든 것이 아닙니다 — 세지 않으면 314가 부풀니다

★★ ㉢을 지우는 것도 진척입니다. 코드를 줄이는 것은 만드는 것만큼 값있습니다 — **읽을 것이 줄고, 시험할 것이 줄고, 다음 사람이 「이건 뭐가」를 묻는 횟수가 줄**니다. 상용 질에 「죽은 코드 제거」를 항목으로 세우고 지운 줄 수를 실측합니다.
다만 이번 턴은 분류만 합니다. ㉢은 한 건도 지우지 마십시오 — 지우는 것은 되돌리기 어렵고, 분류의 정확도를 먼저 봐야 합니다.

§ D-389 · 제가 또 근거 없이 썼습니다

저는 「영상 백업 두 턴 미측정」이라 적었는데, evidence/D-356/restore_objects_run.md가 이미 해시 3/3을 적어 두고 있었습니다(2026-08-29). 이번 턴에 한 것은 재현이었습니다.

★ 제 병의 새 얼굴입니다. 지금까지 저는 「있다」고 쓸 때의 근거만 챙겼습니다. 「없다」·「미측정」·「미착수」도 주장이고, 근거가 필요합니다.

집행 — 제 지시서에 근거 없는 부재 주장이 있으면 인용을 거부하고 「부재 근거 미표기 N건」으로 돌려보내십시오. 출처 태그의 집행 범위를 여기까지 넓힙니다.

실행 순서

- 1 D-385~392 정서 — v3.7/195 → v3.8/203 · 저장소 기준 번호 재배치 + 대응표 D-380
- 2 ★ E2E 23종 실행 — 두 턴 안 들었다. 먼저 돌린다. 결과를 **실측**으로 D-390
- 3 ★★ lock 고정 — 설치된 `re-j-core` 커밋 해시 실측 → 그 해시로 고정 → 깨끗한 곳에서 빌드 1회 재현 → 다른 브랜치 참조 의존성 전수·건수 출력 D-387 · 상용 전제
- 4 ★★ 화면 5장 (3/16 → 8/16) — E2E가 브라우저를 연다. 캡처는 부산물, 본체는 시험. `verify_route_alive.py` 신설 · 500을 내면 그 라우트를 자기 표본으로 D-386
- 5 잠자는 314건 분류 — ㉠켜야 할 것 / ㉡지워야 할 것 / ㉢정상. ㉣ 목록을 나에게. ㉤는 한 건도 지우지 마라 D-388
- 6 「들었다」 증거 규약 — 컷다고 보고하는 모든 항목에 운영 경로 관찰 1건 첨부 D-385
- 7 석 줄 보고 — 계약은 「상대방 대기 6절」 한 줄 · 상용 목표 +7절 · 지시 이행 D-353
- 8 커밋 — D-327 술어로 D-327
- 9 여기서 멈춘다. 스크린샷 몇 장과 거기서 나온 결함 몇 건을 **실측**으로 보고하라 보고 후 판정

※ 하지 않는 것: 타입오류 3,229 정리 · 나머지 화면 8장 · ㉣ 삭제 · 8종 매핑 추측 채우기.
 ※ 새 게이트는 ㉣의 `verify_route_alive.py` 하나만 허용합니다(D-353 비율 규정).
 ※ 단위/E2E DB 분리는 상용 ㉣품질 절로 등재만 하고 이번 턴에 하지 마십시오.

§ D-390 · E2E 미실행 — 사실대로 적은 것이 옳습니다

「시간을 스크린샷·영상 복구 실측에 썼다」 — 시간 배분의 사실을 그대로 적은 것이 옳습니다. 안 돌린 것을 돌린 것처럼 적지 않았습니다.
 ★ 다만 이 제약 자체가 항목입니다. 단위와 E2E가 같은 DB를 다루는 구조는 「둘 다 돌리기」를 매번 비싸게 만듭니다. 분리를 상용 ㉣품질 절로 세우십시오. 지난 턴 메모리에 적은 「동시 실행 금지」는 회피이지 해결이 아닙니다.

대표께 — 수를 정정합니다

① 직전에 보고드린 67.3%를 65.3%로 정정합니다. 그 안에 거짓 초록 두 개가 들어 있었습니다 — ④ 운영에서 「주기를 등록했다」고 보고한 감사·백업이 실제로는 돌지 않고 있었고, ① 기능에서 이벤트 조회 라우트 하나가 운영에서 죽어 있었습니다. 둘 다 이번 턴에 고쳤습니다.

정정 실제 기준으로는 65.3% → 68.2%, +2.9%p입니다.

★ 좋아 보이는 수를 남겨 두지 않았습니다. 틀린 수 위에 다음 판정이 서면 그 판정도 틀립니다.

② 이번 턴 최고의 소식 — 화면을 띄우는 것이 가장 강한 시험이었습니다.

스크린샷을 찍으려고 브라우저를 열자 이벤트 조회 라우트가 500이었습니다. 단위 시험 530건이 전부 초록인 채로 그 라우트는 운영에서 죽어 있었습니다.

3장을 찍으며 죽은 라우트 1건을 잡았습니다. 16장이면 더 나옵니다. 지금까지 저는 스크린샷을 「고객에게 보여줄 것」으로 다뤘는데, 실제로는 「우리가 못 보던 것을 보는 유일한 눈」이었습니다 — 값이 반대편에 있었습니다.

③ 상용 출시를 막는 것 하나가 드러났습니다 — 빌드가 재현되지 않습니다.

프론트엔드 의존성 하나가 커밋이 아니라 브랜치를 가리키고 있습니다. 같은 명령이 시점마다 다른 코드를 가져옵니다 — 오늘 되던 빌드가 내일 깨질 수 있고, 배포할 때마다 다른 것이 나갈 수 있습니다. 타입 오류 3,229건의 뿌리도 이것입니다. 이번 턴 최우선으로 고정합니다.

④ 잡자는 기능은 314건이었습니다(우리 관할 198건). 다만 전부 고칠 것이 아닙니다 — 커야 할 것 / 지워야 할 것 / 원래 정상인 것을 먼저 가릅니다.

★ 죽은 코드를 지우는 것도 진척입니다. 읽을 것이 줄고, 시험할 것이 줄고, 다음 사람이 묻는 횟수가 줄니다. 상용 항목으로 세웠습니다.

⑤ SDN 명세 — 여전히 대기 중입니다. 계약 6조3항 재통지 문안이 필요하시면 바로 만들어 드리겠습니다. 지금 재통지가 필요한 이유는 재촉이 아니라 기록입니다 — 2027년 1월 통합시험에서 「왜 6절이 비어 있는가」를 설명할 때 두 번 보냈다는 사실이 우리를 지킵니다.